

and could be easily translated to design, some specific second-tier requirement might have more than one possible interpretation. The architect's perspective also might not align with the delivery team's, causing absolute chaos later on.

This is more relevant in teams that are located at different geographical locations as clear communication becomes more difficult in these cases. There is a very interesting and somewhat underrated law regarding this as postulated by Michael Conway: "Any organisation that designs a system (defined broadly) will produce a design whose structure is a copy of the organisation's communication structure."

This roughly means that your application's design mirrors the way you communicate across teams within the company! Hence, it is imperative to have clear communication across teams within the company, get the requirements clear down to their grainiest level and agree on the acceptance criteria of the required software well ahead of development.

Zoom out

One of the challenges that most architects face when designing an application from ground up is usually the sheer size of the requirements stack and the delivery commitments associated with it.

Often this ends up with "burst coding" where the development team is given a module to work upon immediately once the requirement is available with the idea that a change would be accommodated as required.

While agile methodologies of development have their merit, none of them advocate developing something without a concrete and stable architecture around it. The first step is then to take a look at the big picture, to zoom out, identify the components that are central to the application and figure out the way they can be connected. Only when you have a clear high-level structural definition of the entire application should development begin.

Choice of technology

You have your design board ready, you know the components involved and the way they are intended to work. Now comes the fun part of choosing technologies to fit your requirements. Some projects have spent weeks and months doing Proof of Concepts (PoCs) on a variety of technologies to find out which fit their needs and arrive at a solution. Simultaneously, others have blindly gone for the industry standard while some have even adopted